



Package Management



archived file

compressed

Managing Software

→ yum → RH
→ apt → Debian

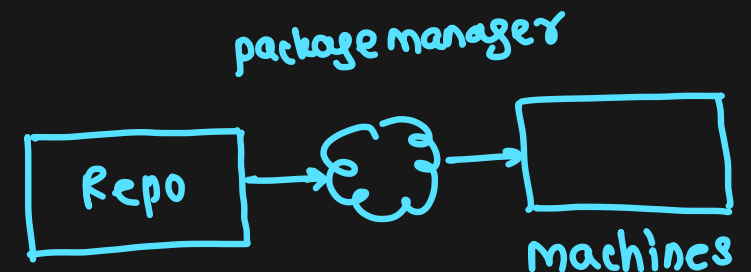


- Maintaining software on Linux systems is usually accomplished with package managers
- Package managers deploy, update, remove, and report on applications
- However, due to the open-source nature of the Linux environment, software applications are often compiled from source code—something that is not common with other operating systems
- Because there are many different Linux distributions available, there are many package managers
- As a general rule, most distributions either originated with Red Hat Linux (and therefore use the Red Hat package managers) or Debian Linux (and therefore use the Debian package managers)

Software Packages

→ archived & compressed file

- Linux software is often distributed in packages that contain everything needed to run the software—executables, configuration files, supporting resources, and documentation → manual pages
- Therefore, the process of installing, maintaining, and removing software is referred to as package management, and the tools required to maintain software are package managers
- One critical component of package managers is the ability to report software information
 - Package managers track software installation information in a database that can be queried
 - Such queries might include a list of all installed software, a list of software versions, or specific information on a single software package
 - The ability to query the package manager's installation database provides sysadmins with essential information on the system and its applications
- A second critical part of package management is the source from which package managers pull the software
 - A storage location for software packages is referred to as a repository → collection of packages
 - Sysadmins control what repositories are available to package managers
- Repositories can be located in a few key areas:
 - An internal server containing only authorized packages
 - A distribution vendor, such as Red Hat or Ubuntu
 - A public package repository



Red Hat vs Debian

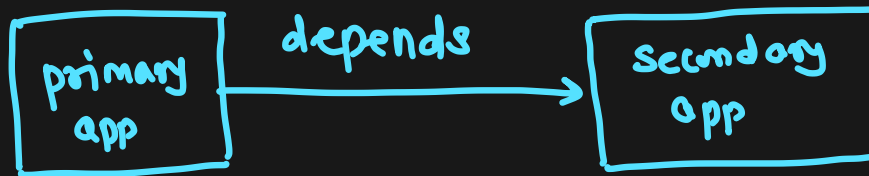


- The Red Hat and Debian package managers handle the software lifecycle in much the same way
- Each approach has a method for installing, updating, removing, and reporting software on the system
- Package managers track versions, the location of installed files, and place documentation files in standard locations
- Each package manager has its own related commands, and those commands cover the three major parts of the software lifecycle (installation, maintenance, and removal)
- **Red Hat Package Managers**
 - Red Hat Package Manager (RPM) → low level package manager ← uses
 - Yellow Dog Update Manager (YUM)
 - Dandified YUM (DNF) package manager } high level ← —
- **Debian Package Managers**
 - dpkg package manager → low level ← uses
 - Advanced Packaging Tool (APT) → high level ←
- **Suse Zypp package manager**
 - SUSE Linux uses its own package manager for software maintenance: Zypp
 - The ZYpp package manager is the basis for the GUI tool YaST (Yet another Setup Tool) and the command-line zypper utility

Software Dependencies



- Package managers make software maintenance much more straightforward
- One challenging part of handling Linux software, however, is dependencies
- Some software packages rely on other packages to function
- If the software an application depends on is not installed, it will either fail entirely or not work as intended
- Early package managers required administrators to manually provide the dependent applications before installing the primary package
- Modern package managers such as YUM, DNF, and APT install supporting applications automatically, ensuring that a single installation command provides the user with everything needed to run the program
- Periodically, package dependency issues still arise, so recognize the problem (failed dependency) and the solution (install the packages required for the primary application to run)





Red Hat Package Manager

rpm

yum | dnf

RPM package → .rpm



- An RPM package is essentially an archive file that contains:
 - Compiled software binaries → executable(s)
 - Configuration files → /etc/
 - Metadata (name, version, description, dependencies)
 - Installation scripts (pre-install, post-install, pre-uninstall, post-uninstall)
 - File manifests (list of all files included)
 - Digital signature (GPG)
- Features of rpm packages
 - ✓ Dependency management: RPM tracks what packages require what libraries. If you try to install a package without its dependencies, RPM will show an error
 - ✓ Verification: RPM can verify that installed files haven't been tampered with → GPG Key
 - ✓ Digital signature: RPM packages are GPG-signed to ensure authenticity and integrity
 - ✓ Database tracking: RPM maintains a local database (/var/lib/rpm/) that tracks:
 - All installed packages
 - All files belonging to each package
 - Package metadata

Red Hat Package Manager



■ RPM → Does not resolve dependencies automatically

- The Red Hat Package Manager (RPM) is the oldest of the package managers
- It provides good basic functionality with a simple syntax and a lot of flexibility
- It maintains a software database for package queries
- One of its main drawbacks is package management, which is not as robust as YUM and DNF

→ low level
package manager

■ YUM

- Yellow Dog Update Manager (YUM) relies on .rpm packages and also tracks software in an RPM database
- Like RPM, YUM alerts users to software dependencies
- However, it also offers the ability to automatically install dependencies so that a single YUM command installs the specified software and all required software for it to function

■ DNF

- Both RPM and YUM are superseded by Dandified Yum (DNF), which includes all the same functionality as both with additional plugins and features
- Typically, any Linux system that uses .rpm software packages recognizes RPM, YUM, and DNF commands

RPM Commands



- The use of Red Hat package managers is primarily focused on installing, updating, upgrading, and uninstalling software packages
- These packages have a .rpm file extension
- RPM files are pre-compiled and usually include all supporting documentation and configuration files
- The idea is that a single package contains the entire application
- Commands
 - rpm -ivh <package>
 - rpm -Uvh <package>



Using RPM queries

- rpm is the legacy command to manage RPM packages
- Do Not use rpm to install packages as it does not consider dependencies
- rpm is still useful though to perform package queries
- rpm queries by default are against the database of installed packages
 - `rpm -qf /any/file`
 - `rpm -ql mypackage`
 - `rpm -qc mypackage`
 - `rpm -qp -scripts mypackage-file.rpm`



Basic package management

- yum install <package>: install a package
- yum update <package>: update a package
- yum update -security: apply only security-related updates
- yum upgrade: upgrade packages considering obsolete
- yum downgrade <package>: downgrade a package
- yum remove <package>: remove or uninstall a package
- yum autoremove: remove unnecessary dependencies
- yum reinstall <package>: reinstall current version of package
- yum history: show yum history

Query and Search



- yum list installed: list all installed packages
- yum list available: list all packages available to install
- yum list all: list installed and available packages
- yum list updates: list available updates
- yum info <package>: display detailed information about a package
- yum search <keyword>: search for packages by name or description
- yum provides <file_path>: find which package owns a specific file
- yum deplist <package>: list the package's dependencies
- yum check-update: check updates without installing them

Managing yum groups



- yum groups list: show the list of groups available
- yum groups info <group name>: show selected group's information
- yum groups install <group name>: install a group

Repositories



- When the rpm, yum, or dnf commands are issued to install a package, the commands must know where to look for the software
- If a package has been downloaded to a specific file, the package manager can be directed to that file with the absolute path
- However, locally downloaded and stored software is not recommended
- Version control is difficult, as is enforcing standard applications
- Therefore, administrators should define centralized repositories of packages and configure the YUM and DNF package managers to use those repositories when installing or updating software
- Types
 - Local repositories: These repositories are stored on the system's local storage drive. Installation is easy, but version control is difficult because of the decentralized nature of local repositories. If you manage 20 Linux servers, each would have its own repository to be maintained
 - Centralized internal repositories: These repositories are stored on one or more systems within the internal LAN and managed by the Linux administrator. This centralized approach makes version control much simpler. If you manage 20 Linux servers, one could host the repository and the other 19 could download their packages from it
 - Vendor repositories: These repositories are maintained on the Internet, often by the distribution vendor. Version control is very difficult because the vendor decides what package versions are made available



Repository File structure

- The .repo files are stored in the /etc/yum.repos.d/ directory
- Some of the components of the .repo file include:
 - [repo-name] —The repository name.
 - name=Repository Name —The human-friendly name of the repo.
 - baseurl= —The path to the repo. May be a file (file:///) or http:// path.
 - enabled=1 —Enables the repo.
 - gpgcheck=0 —Disables GPG checking
- yum repolist: shows the list of repositories



Debian Package Manager

dpkg

apt

Debian package



- A Debian package is the standard software distribution format for Debian and its derivatives (Ubuntu, Linux Mint, etc.)
- These packages, which use the .deb file extension, contain compiled software along with metadata that enables the package management system to install, upgrade, and remove software efficiently
- The Debian package management system, called dpkg, forms the foundation of software management across the entire Debian ecosystem
- Two types of debian packages
 - Binary Packages
 - Binary packages (.deb files) contain pre-compiled executables, libraries, configuration files, documentation, and metadata
 - These are what users typically install on their systems. The binary package includes everything needed to run the software on a specific computer architecture
 - Source Packages
 - Source packages consist of three components :
 - .dsc file (Debian Source Control) - Contains metadata about the source package
 - .orig.tar.gz - The original upstream source code
 - .debian.tar.xz - Debian-specific modifications and packaging files
 - Source packages allow users to rebuild the software from scratch or inspect how the Debian maintainers customized the original software

→ name / description / dependencies

Debian package manager



- The Debian package management system is one of the most mature and robust software management frameworks in the Linux ecosystem
- It uses .deb packages as its core format and employs a multi-layered approach to handle software installation, removal, updating, and dependency resolution
- This system forms the foundation not only for Debian itself but also for its derivatives including Ubuntu, Linux Mint, and many others

Core Components



■ dpkg - The Low-Level Package Manager

- dpkg (Debian Package) is the fundamental, low-level tool that directly manipulates .deb package files
- It installs, removes, and queries packages but does not automatically handle dependencies
- Think of it as the engine that performs the actual installation work
- Key characteristics of dpkg:
 - Works directly with .deb files
 - Maintains a local database in /var/lib/dpkg/
 - Provides powerful querying capabilities
 - Requires manual dependency resolution
 - Essential for low-level operations and troubleshooting

■ APT - The Advanced Package Tool

- APT is the high-level package management system that builds on top of dpkg. It automates the entire process by:
 - Automatically resolving and installing dependencies
 - Managing remote software repositories
 - Handling package updates and upgrades
 - Providing secure package verification via GPG signatures

Basic package management

can also use "apt-get" → old interface



- apt update: Refresh package lists from repositories
- apt list --upgradable: Show packages that can be upgraded
- apt install package: Install a package with dependencies
- apt remove package: Remove package (keep configs)
- apt purge package: Remove package and all config files
- apt autoremove: Remove unnecessary dependencies
- apt upgrade: Upgrade all installed packages
- apt full-upgrade: Upgrade with intelligent dependency handling

Query and searching packages



- apt search keyword: Search for packages by keyword
- apt show package: Display detailed package information
- apt list --installed: List all installed packages

Repositories



- Debian repositories are configured in two locations:
 - /etc/apt/sources.list - Main configuration file
 - /etc/apt/sources.list.d/*.list - Directory for additional sources

Additional tools and frontends



- **Aptitude:** Text-based ncurses interface with advanced dependency resolution
- **Synaptic:** Graphical package manager with intuitive GUI
- **Gdebi:** Simple GUI for installing local .deb files
- **Tasksel:** Tool for installing predefined task collections
- **Apt file:** Search for files in uninstalled packages